

shows that the analysis was passed and zero bugs, zero vulnerabilities, and one security hotspot were reported. There are four code smells and fifteen minutes of technical debt.

Some other details of bugs and vulnerabilities are shown in Figure 12. The left panel shows the categories like critical, major, minor, info, etc. This panel can be used for viewing different issues according to severity and type. There are other tabs like resolution, status, security category, etc, which can be used for viewing the ways of resolving issues, their status, category, and so on.

Results and discussion

Both Snyk and SonarQube generate a report after SAST analysis, on the basis of which we can update the code to resolve bugs and vulnerabilities. So, it is important to know how to use this report for best results. The report generally categorises the issues into 'major' and 'minor'. Major issues need urgent assistance, while minor issues are mild issues that can be at a lower priority. The report displays the bugs, vulnerabilities, and security hotspots. These are the security-sensitive pieces of code that the developer needs to review manually. Code smells are listed to show the maintainability issues with the code. Technical debt or debt, which is the estimated time for reviewing and maintaining the code, is also shown. The report also gives ways to fix the issues, making life easier for any tester or developer.

With this article, we understood the importance of security in applications and of Static Application Security Testing or SAST in DevSecOps. We understood that various tools are available for SAST; for demonstration, we have used Snyk and SonarQube. These tools are open source, can be used for applications in various languages and provide user-friendly functions. We have used Snyk in two ways, and applied SonarQube Analysis on a Java and a Python application. We also understood some important terms in the report, using which we can fix the security problems in the code. 

```
INFO: SCM Publisher No SCM system was detected. You can use the 'sonar.scm.provider' property to explicitly specify
INFO: CPD Executor Calculating CPD for 3 files
INFO: CPD Executor CPD calculation finished (done) | time=12ms
INFO: Analysis report generated in 112ms, dir size=9 KB
INFO: Analysis report compressed in 37ms, zip size=19 KB
INFO: Analysis report uploaded in 9ms
INFO: ANALYSIS SUCCESSFUL, you can browse http://localhost:9000/dashboard?id=feedback
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis
INFO: Here about the report processing at http://localhost:9000/api/ce/task?id=AXT0CckJmJpweLPQg
INFO: Analysis total time: 16.532 s
INFO:
INFO: -----
INFO: EXECUTION SUCCESS
INFO: Total time: 18.958s
INFO: Final Memory: 134M/454M
INFO: -----
shriya@shriya-Lenovo-IdeaPad-318-152KB:~/Documents/UlltFeedback-masters$
```

Figure 10: Successful analysis of Python project

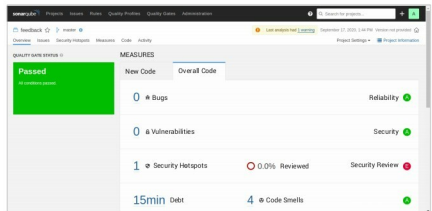


Figure 11: Detailed analysis of Python project

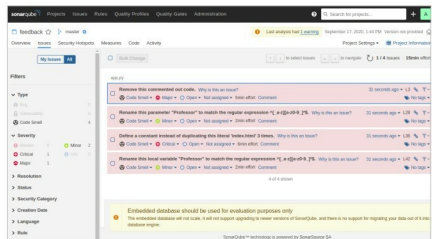


Figure 12: Bugs and vulnerabilities

References

- [1] 'SonarQube Concepts', <https://docs.sonarqube.org/latest/user-guide/concepts/>
- [2] 'SonarQube Server Installation', <https://docs.sonarqube.org/latest/setup/get-started-2-minutes/>.
- [3] 'SonarQube', <https://docs.sonarqube.org/latest/analysis/scan/sonarscanner/>.
- [4] 'Snyk', <https://snyk.io/>.
- [5] 'Dockerhub', https://hub.docker.com/_/sonarqube/.

By: Shriya Kabra and B. Thangaraju

The authors are associated with the Open Source Technology Lab at the International Institute of Information Technology, Bengaluru.